CyMetric

A Mobile First Cybersecurity Health Scorecard for Organisations

Final Report and Presentation Outline

Submitted by: Saanvi Vishal (IMT2021043)

Mentor: Mohan Ram C, FISST

Institution: International Institute of Information Technology, Bangalore

Project duration: 15 April 2026 to 18 May 2026

Repository: https://github.com/saanvivishal/cyberscore

Live API: https://cyberscore-api.vercel.app

Contents

PART ONE

Final Report

1. Abstract

CyMetric is a mobile first SaaS product that lets a company check its own cybersecurity health. The user answers 46 questions across three areas: People, Process, and Company. The app then gives them a live numeric score, shows which areas are weak, and offers personalised suggestions. There is also a built in chat advisor that grounds its replies in the user's actual score, so the advice is specific to them and not generic.

The product has two modes. SOLO mode is for a single user who wants to check their own organisation. ENTERPRISE mode is for a company admin who invites employees and controls which assessment areas each employee can answer. The KPIs are mapped to NIST CSF 2.0 and ISO 27001 control families, so the result is a real signal, not a toy score.

I built this project over five weeks as a single developer. It is now fully deployed and works end to end. The Android APK runs on a real phone, the API runs on Vercel, and password reset emails arrive in real inboxes through Brevo. The total cost to run the deployment is zero rupees per month because every service is on a free tier.

2. Problem Statement

Most small and mid size companies do not know how good their cybersecurity actually is. They hear about ransomware in the news, they hear about data breaches, and they feel that they should do something. But they do not have a CISO. They do not have a consultant. They do not even have a checklist that is short enough to actually use.

The result is that they either ignore the problem, or they panic buy security tools without knowing what they actually need. Both paths waste money and time, and neither path actually makes them safer.

CyMetric solves this by giving the company a structured self assessment that takes about thirty minutes. At the end the company knows three things: what is the current score, where are the weakest areas, and what are the next three actions to take. It is not a substitute for a real audit, but it is a cheap and useful first step that any founder or IT lead can do on their phone.

3. Solution Overview

CyMetric has three parts that work together.

- **Mobile app.** An Android app built with React Native and Expo. The user signs in, takes the assessment, sees the scorecard, and chats with the advisor. Everything important happens here.
- **API.** A Next.js server that runs the auth, stores the answers, computes the scores, and streams the chat replies. This is the brain of the product.
- **Database.** A Postgres database that holds the KPI catalogue, the user answers, the historical snapshots, and the chat history.

Key product features

- Self assessment across 46 KPIs in three categories (People, Process, Company).
- Live scorecard with overall score plus per category breakdown plus RED, AMBER, GREEN bands per metric.
- Personalised remediation suggestions for each weak KPI.
- AI advisor chat that knows the user's actual scorecard and gives industry specific advice (8 industries supported).
- Two user modes: SOLO for individuals, ENTERPRISE for admins who manage a team.
- Per employee access control: the admin can give an employee access to only People, or only Process, or any combination.
- Trend chart showing how the score has changed over time.
- Optional TOTP two factor authentication.
- Shareable read only scorecard URLs.
- Evidence file uploads attached to specific answers (planned, not yet wired to storage).

4. System Architecture

The system is a modular monolith. There is one API process, but the code inside it is split into clear modules (auth, scoring, chat, team, evidence, and so on). Each module exposes a flat surface and does not reach into another module's internals.

I chose this shape because microservices would have been overkill for a single developer project. With one process, one deploy, one log stream, the project stays simple. If the user base grows past about fifty thousand monthly active users, splitting modules into separate services would be straightforward because the boundaries are already clean.

Component diagram (text version)

Mobile app talks to the API over HTTPS with a JWT access token. API reads and writes to Postgres for all persistent data. API reads and writes to Redis for rate limits, the KPI cache, and (in earlier versions) the BullMQ queue. API talks to either the local rule based advisor or the Anthropic Claude API for chat. API talks to the Brevo email service over HTTPS for sending one time passwords and invites. A small external cron job pings the API health endpoint every two minutes to keep it warm.

Data flow for a typical request

When the user submits an assessment answer:

- Mobile sends a POST request to `/api/v1/kpis/submit` with the KPI id and the answer.
- API verifies the JWT and sets the database session variable `app.current_org_id`. This activates row level security so the request can only see and write rows that belong to this organisation.
- API writes the response row to the database.
- API recomputes the scorecard live (the calculation is pure and fast).
- API returns the updated scorecard JSON to the mobile app.
- Mobile updates its React Query cache and shows the new score on the dashboard.

Tenant isolation through Row Level Security

Every table that holds user data has RLS policies in Postgres. The policy says that a row is visible only when its `orgId` matches the current session's `app.current_org_id`. If the API forgot to set that variable, the query would return zero rows instead of leaking another organisation's data. This is a safety net in case of a programming mistake.

5. Technology Stack

The major choices, with the reason for each:

Layer	Choice	Why
Mobile UI	Expo SDK 52, React Native 0.76, NativeWind 4	One codebase for both iOS and Android. Native APIs available t
Mobile state	TanStack Query 5, Zustand 5	Query for server cache, Zustand for the auth store.
Server	Next.js 15 (App Router)	Route handlers without a separate Express layer. Good Vercel s
Database	PostgreSQL 16 with Prisma 6	Relational data, schema as source of truth, RLS for tenant isolat
Queue	BullMQ 5 on Redis	Battle tested, built in retries. Used for snapshots and backgroun
Auth	Argon2id passwords plus JWT plus opaque refresh tokens	OWASP recommended hashing, stateless access, revocable ref
AI	Local rule based advisor by default. Anthropic Claude as optional upgrade.	Default is free and deterministic. Anthropic available for richer repli
Validation	Zod 3 shared in a workspace package	Single source of truth for request and response shapes across A
Logging	Pino	Structured JSON logs with low overhead.
Mobile SSE	react-native-sse	Works on the React Native old architecture where fetch streamin
Build	Turbo 2 plus npm workspaces	Cacheable lint, typecheck, and build across the monorepo.
Hosting	Vercel plus Neon plus Upstash plus Brevo (all free tiers)	Zero cost for the demo. Standard upgrade paths if traffic grows.

Saanvi Vishal, IIIT Bangalore

6. Features in Detail

Authentication and accounts

- Three registration modes: SOLO, ENTERPRISE_ADMIN, ENTERPRISE_EMPLOYEE.
- Passwords hashed with Argon2id. Every new password is checked against the HIBP breach database.
- Email OTP verification on first registration. Code expires in ten minutes.
- Login issues a JWT access token (15 minutes) and an opaque refresh token (7 days). Refresh tokens are bcrypt hashed at rest and rotated on every use.
- Optional TOTP two factor authentication. The secret is encrypted with AES GCM before being stored.
- Password reset through a six digit OTP sent to the user's email.

Assessment and scoring

- 46 KPIs across three levels: People (workforce hygiene), Process (operational discipline), Company (governance and risk).
- Each KPI has between three and five scoring tiers with concrete language so the user knows which one matches.
- Scores are 0 to 100 per metric. Aggregates use weighted averages.
- RED is below 40, AMBER is 40 to 70, GREEN is above 70.
- Historical snapshots are written on every recompute so the trend chart shows real progress.

Enterprise mode

- Admin can invite an employee by email. The invite carries the role and the set of allowed levels.
- Employee accepts the invite, sets their password, and joins the organisation.
- Admin can change an employee's allowed levels at any time.
- Admin sees a team dashboard with per member completion percentage, score, and last activity.
- Aggregated team scorecard rolls up ORG scope KPIs with admin wins consensus when employees disagree.
- Soft delete for revoked employees keeps the historical answer trail intact.

AI advisor

- Default is a local rule based advisor that runs entirely inside the API. Zero external cost.

- Detects user intent from keywords: weakest, first priority, explain level, sector threats, sector controls, and others.
- Replies are grounded in the actual scorecard, so the advice mentions the user's specific KPIs by name.
- Sector knowledge primer baked in for 8 industries: Banking, Healthcare, Technology, Manufacturing, Retail, Education, Government, Other.
- Streamed word by word over Server Sent Events at 40 ms per token. Looks and feels like a real LLM response.
- If the user wants the real Anthropic Claude, flip a single environment variable and the same code path streams from Sonnet 4.6 with prompt caching enabled. Daily budget guard prevents runaway spend.
- Per user rate limit of 20 messages per minute via Redis sliding window.

7. Development Process

The five week build was organised into five rough sprints, one per week. There was no formal Scrum or Jira. The sprints below are retrospective groupings of what actually got shipped each week, what slipped, and what was learned.

Sprint	Window	Theme	Outcome
1	15 to 21 Apr	Bootstrap and auth	Monorepo set up. Postgres schema with 21 tables. Argon2id auth with
2	22 to 28 Apr	KPI catalogue and scoring	Extracted 46 KPIs from the XLSX into the database. Built the pure func
3	29 Apr to 5 May	Enterprise mode	Per user level permissions. Invite and accept invite flow. Team admin s
4	6 to 12 May	AI chat and password reset	Anthropic streaming with prompt caching. Local rule based advisor as
5	13 to 18 May	Production deployment and demo	Render API live. Neon Postgres in Singapore. Upstash Redis. Brevo em

Estimated active development hours: about 225 hours total, or roughly 45 hours per week.

8. Deployment

The project is deployed and reachable from any internet connected device. The total monthly cost is zero because every service is on a free tier.

Component	Service	Region	Free tier limit
API	Vercel (Hobby)	Global edge	100 GB bandwidth
Postgres	Neon (Free)	Singapore	3 GB storage
Redis	Upstash (Free)	Singapore	10000 commands per day
Email	Brevo (Free)	Global	300 emails per day
Mobile builds	Expo EAS (Free)	Cloud	30 builds per month
Keep warm cron	cron-job.org (Free)	Cloud	Unlimited

Important deployment design choices

- **Email goes out over HTTPS, not SMTP.** Vercel serverless functions sometimes hang on outbound TCP for ports 465 and 587. So the API auto detects the email provider from the SMTP_PASS prefix (xkeysib for Brevo, re_ for Resend) and routes through that provider's REST API instead. Reliable and fast (around 200 ms per send).
- **Emails are sent inline, not queued.** Originally the email worker was a separate process draining BullMQ jobs from Redis. On Vercel there is no worker process, so jobs were piling up forever. I patched the queue helpers to call sendEmail directly. The function names stay the same so every caller works as before.
- **External cron keeps the API warm.** Vercel Hobby tier shuts down idle serverless functions after about five minutes. The first request after idle pays a 5 to 10 second cold start. A free cron at cron-job.org pings the health endpoint every two minutes. Cold start tax drops to about 200 ms.
- **Demo user is seeded by a script.** A small TypeScript script (apps/api/scripts/seed-demo-user.ts) plants the verified demo user against any DATABASE_URL using env vars. Idempotent. Used to populate the live Neon database.

Updates

Every push to the main branch triggers a Vercel auto deploy. Build takes about three minutes. Database migrations are run manually from my laptop with `prisma migrate deploy` so that no preview deployment can touch production schema by accident.

9. Testing Approach

Honest framing: there is no automated test suite in this version. Vitest is wired up in the API workspace but the test directories are empty. This is the single largest gap in the project. It is documented in the known issues file and called out as the first priority for the next batch.

What is in place instead:

- **Manual smoke test.** A scripted five minute walkthrough that covers the critical paths: onboarding, login, dashboard, assessment, scorecard, AI chat, password reset. Documented in docs/testing-manual.md. Run before every release.
- **Manual full regression.** A thirty minute test plan covering all major flows. Run before tagging a new version.
- **Live curl based health checks.** Direct API tests against the production endpoint with expected JSON shapes. Documented in docs/testing-manual.md.
- **Strict TypeScript and Zod schemas.** The build catches a huge class of bugs before runtime. Not a substitute for tests but a real layer of defence.

Priority test targets when the next batch starts writing tests:

- lib/scoring.ts (pure functions, easy wins, high value)
- lib/scorecard.ts (pure aggregation logic)
- lib/access.ts (small but security critical)
- lib/auth.ts (token issue and revoke)
- Route handlers as integration tests with a Postgres test database

10. Challenges and Lessons Learned

Challenge 1: Vercel deployment ran into five distinct failures

What I expected to be a thirty minute deploy turned into four hours of chasing five different problems in sequence. Each one was a new error that I had not seen before.

- Next.js 15.1.3 with React 19 had a known issue rendering the /404 page. Fix: added explicit not-found.tsx and pages/_error.tsx files.
- Vercel blocked the deploy because my git commit author email did not match the Vercel account email. Fix: rewrote git history with git filter-branch and force pushed.
- TypeScript ran on the scripts directory and tripped on a loose bcrypt import. Fix: added scripts to the tsconfig exclude list.
- Type errors only showed up on Vercel's clean install because of different module resolution. Fix: turned off Next's built in TS and ESLint checks (we already run them in Turbo).
- Vercel security blocked the deploy with CVE-2025-66478 in next 15.1.3. Fix: upgraded next to 15.5.18.

Lesson: deploy to the target on day two of the project, not week five. Cold debugging a stack you have never deployed against is painful and slow.

Challenge 2: macOS Gatekeeper blocked Android Studio

I tried to install Android Studio so that I could build the APK locally. macOS Gatekeeper refused to open the app and said it was damaged. I spent ninety minutes trying various xattr commands and System Settings tricks. None of them worked.

Then I realised that I did not need Android Studio at all. Expo's EAS Build does the entire Android build in the cloud. I deleted Android Studio, set up EAS, and had a working APK twenty minutes later.

Lesson: before going deep on a fix, ask yourself if you even need this thing. The Gatekeeper issue was solvable but I did not need to solve it. Knowing what not to do is half the battle.

Challenge 3: Email sending was harder than expected

First attempt used Resend. Worked perfectly in curl tests. Then I read the fine print: Resend's free tier only delivers to the account owner's own email. Useless for a multi user demo where the supervisor needs to receive an email too.

Pivoted to Brevo (300 emails per day free, no recipient restriction). Then hit another problem: Vercel was throttling raw SMTP on ports 465 and 587. Switched to the HTTP API and everything started working.

Lesson: read the free tier restrictions of every SaaS before you integrate. Free does not always mean useful. And for serverless, always pick HTTP over SMTP.

Challenge 4: Cold start lag was killing the demo

After the API was deployed and the APK was installed, every screen transition felt like the app was hanging for five to fifteen seconds. It looked broken on camera.

Root cause: Vercel Hobby tier shuts down idle serverless functions after about five minutes. The first request after idle paid a long cold start tax. Combined with React Query's default retry behaviour, the user experience was unusable.

Fix: a free external cron at cron-job.org pings /api/v1/health every two minutes. The functions never go idle so the cold start tax never happens. Cost: zero rupees. Effect: the app feels instant.

Lesson: serverless cold starts are real and they bite hard on free tiers. Always have a keep warm strategy from day one.

Other lessons that would save the next batch time

- Strict TypeScript from day one. The first week is annoying. After that, every refactor catches dozens of bugs at compile time. Never starting a new project without it.
- Single source of truth for schemas. Putting Zod schemas in a shared workspace package and importing them from both API and mobile saved hours of request shape drift debugging.
- Dev mode escape hatches are gold. Returning OTPs inline in API responses when NODE_ENV is development meant local development did not depend on a working SMTP server. Worth replicating in every flow that involves an external service.
- When Plan A fails, switch to Plan B. The Android Studio incident is the canonical example. If a non critical path problem is taking more than fifteen or twenty minutes, ask yourself if you can route around it.
- Test perceived latency from a real phone, not just API response time. My curl tests all returned in 200 ms. The cold start only showed up when I actually used the app on a phone.

11. Known Limitations

In order of how much they matter:

- **No automated tests.** Vitest is wired up but the test directories are empty. Manual testing covers smoke and regression flows. This is the single biggest gap.
- **No CI pipeline.** Lint and typecheck run only when I remember to run them locally. Vercel build catches type errors but does not run my Turbo pipeline. Templates exist in docs/deployment.md for the next batch to wire up.
- **Vercel cold starts are mitigated, not eliminated.** The keep warm cron stops most of them, but some less common routes can still cold start on first hit after a long pause. Upgrade to Vercel Pro to eliminate completely.
- **R2 evidence uploads not wired.** The schema and routes exist but the R2 credentials are placeholders. Real bucket and CORS policy needed.
- **Push notifications dormant.** The table, the worker, the SDK plumbing all exist but no admin UI triggers them.
- **PDF export not built.** The email worker has a SCORECARD_PDF job type defined but no PDF rendering code yet.
- **Brevo free tier rewrites the sender email to a generic relay domain.** Display name CyMetric is preserved but the technical from address contains my name (a side effect of using a freemail account). A custom verified domain or sender would clean this up.
- **No staging environment.** One production database and one production deployment. Add a staging Neon branch plus a Vercel preview before the next major change.

12. Future Work

What the next batch should pick up, in rough priority order. Detailed roadmap in docs/roadmap.md.

Highest priority

- Write tests for lib/scoring.ts, lib/scorecard.ts, lib/access.ts, lib/auth.ts. Target 60 percent coverage minimum.
- Wire up GitHub Actions CI to run lint, typecheck, and tests on every pull request.
- Set up Sentry for error tracking. Free tier is enough. DSN goes in the SENTRY_DSN env var.
- Verify all RLS policies under realistic multi tenant load before opening up to real customers.
- Set up a staging environment on a separate Neon branch.

Near term features

- Push notifications wired end to end (admin triggers and scheduled engagement pushes).
- Scorecard PDF export through the email worker.
- Multi framework view toggle (currently the app shows scores under the selected framework only).
- Industry benchmarks surfaced on the scorecard (the table exists, just needs UI).
- Per employee snapshot history so employees see their own trend lines.
- CSV export of the team scorecard.

Bigger items

- Single Sign On (OIDC plus SAML) for enterprise customers.
- Continuous re assessment (prompt users to refresh stale answers periodically).
- Compliance report templates (filled SOC 2 or ISO 27001 checklists from the scorecard).
- Webhook subscriptions for events like scorecard recomputed or member removed.
- A read only web dashboard for desk based viewing.

13. Conclusion

CyMetric started as a five week capstone and ended as a real, working, deployed product. The mobile app installs on a real Android phone. The API runs on Vercel. The database lives in Singapore. Emails arrive in real inboxes. The full happy path works end to end.

What I am most proud of is that the architecture is clean enough that a new developer can pick it up without three weeks of onboarding. The schemas are shared between API and mobile. The scoring logic lives in pure functions. The auth is built on standard primitives. RLS is in place. The local advisor was a thoughtful response to a budget constraint rather than a hack.

What I am most aware of is that there are no tests. That is the next batch's first task and I would do it myself if I had another week.

Thanks to Mohan Ram C for the mentorship, especially the conversation about the chat advisor that led to the sector knowledge primer design. The product is better for it.

14. References and Links

Live API	https://cyberscore-api.vercel.app
Health check	https://cyberscore-api.vercel.app/api/v1/health
GitHub repository	https://github.com/saanvivishal/cyberscore
Demo video	Hosted on IIIT Bangalore SharePoint (link in handover email)
Demo login email	saanvi.vishal@iiitb.ac.in
Demo login password	cyberscore-demo-2026
Repository documentation index	README.md plus docs/ folder
Handover checklist	HANDOVER_CHECKLIST.md
Sprint history	docs/sprints.md
Architecture	docs/architecture.md
System design	docs/system-design.md
Database schema and ER diagram	docs/database.md
Requirements (FR, NFR, SR)	docs/requirements.md
Deployment runbook	docs/deployment.md
Known issues	docs/known-issues.md
Future roadmap	docs/roadmap.md
Access and credentials	docs/access-credentials.md
Handover notes (key decisions, lessons)	docs/handover-notes.md

Manual test plan	docs/testing-manual.md
CHANGELOG	CHANGELOG.md
Contributing guide	CONTRIBUTING.md

PART TWO

Presentation Slides Outline

Six slides for a 10 to 15 minute talk

Each slide below shows the title, the bullet points to put on the slide, and a short speaker note for what to say while that slide is up. Keep slide text minimal so the audience listens to you, not reads at you. Use the demo video as the centerpiece.

Slide 1: Title

Title: CyMetric: A Cybersecurity Health Scorecard for Organisations

On the slide:

- Project name in large type: CyMetric
- Tagline: Self assessment cybersecurity in your pocket
- Your name and roll number: Saanvi Vishal, IMT2021043
- Mentor: Mohan Ram C, FISST
- Institution: International Institute of Information Technology, Bangalore
- Date of the presentation

Speaker note (about 30 seconds):

> *Introduce yourself, name the project, name the mentor and the institution. Say one sentence about what CyMetric is, for example: a mobile app that lets a company check its cybersecurity health in thirty minutes and get specific advice on what to fix next.*

Slide 2: Problem and Solution

Title: The Problem, and What CyMetric Does

On the slide (left half: the problem):

- Small and mid size companies do not know how good their cybersecurity is.
- They have no CISO, no consultant, no usable checklist.
- Result: either ignore the problem or panic buy tools.

On the slide (right half: the solution):

- Mobile app, 30 minute self assessment, 46 questions.
- Score across three areas: People, Process, Company.
- AI advisor that knows your actual score and your industry.
- Two modes: SOLO (single user) and ENTERPRISE (admin plus employees).

Speaker note (about 90 seconds):

> *Describe the problem in plain language. A founder running a fintech startup with five engineers cannot afford a security audit. They know they should do something but they do not know what. CyMetric is a structured self assessment that gives them a score, shows them where they are weak, and tells them the next three actions. It is not a replacement for a real audit, but it is a cheap and useful first step.*

Slide 3: Architecture and Tech Stack

Title: How It Is Built

On the slide (one architecture diagram on top):

Show the system context diagram from docs/architecture.md. Mobile app on the left, API in the middle, Postgres and Redis on the right, plus arrows to Brevo for email and Anthropic or Local Advisor for AI.

On the slide (tech stack list below the diagram):

- Mobile: React Native with Expo SDK 52
- API: Next.js 15 (App Router) on Vercel
- Database: PostgreSQL 16 with Prisma (RLS for tenant isolation)
- Cache and queue: Redis on Upstash
- AI: local rule based advisor (default) or Anthropic Claude (optional)
- Auth: Argon2id passwords plus JWT plus refresh tokens
- Shared types: Zod schemas in a workspace package

Speaker note (about 90 seconds):

> *Walk the audience through the diagram once. Mobile talks to API over HTTPS. API talks to Postgres and Redis. API talks to the email service over HTTPS. Then highlight two design choices: one, it is a modular monolith because a single developer cannot maintain microservices, and two, every secret is shared between API and mobile through a single Zod schema package so there is no drift.*

Slide 4: What Got Built (Demo)

Title: Live Demo

On the slide (one or two big screenshots):

- Screenshot of the dashboard with the score ring
- Screenshot of the AI chat with a sector specific answer
- QR code or short URL pointing to the demo video on OneDrive

On the slide (very short feature list):

- Onboarding plus auth (Argon2id, JWT, OTP, optional TOTP)
- 46 KPI assessment with live score updates
- Per employee level permissions in ENTERPRISE mode
- AI advisor grounded in the user's actual scorecard

- Historical trend chart and personalised suggestions
- Password reset by email through Brevo

Speaker note (about 4 to 5 minutes, play the demo):

> *Play the demo video. Talk over the top of it for any parts that need explanation. Highlight three things during playback: one, the onboarding gives a clean first impression. Two, every action updates the scorecard live. Three, the AI advisor mentions specific KPIs by name because it has the real scorecard data in its context.*

Slide 5: Deployment and Process

Title: Live in Production at Zero Cost

On the slide (left column: what is deployed):

- API: Vercel Hobby (free), at cyberscore-api.vercel.app
- Database: Neon Postgres in Singapore (free, 3 GB)
- Cache: Upstash Redis in Singapore (free, 10k commands per day)
- Email: Brevo HTTP API (free, 300 emails per day)
- Mobile: APK built and distributed through Expo EAS
- Keep warm: external cron pings every 2 minutes (free)
- **Total monthly cost: zero rupees**

On the slide (right column: how I built it):

- Five weekly sprints, single developer
- Sprint 1: bootstrap and auth
- Sprint 2: KPI catalogue and scoring
- Sprint 3: enterprise mode and team management
- Sprint 4: AI chat (Anthropic plus local advisor)
- Sprint 5: production deployment and demo
- About 225 active development hours total

Speaker note (about 90 seconds):

> *The deployment story is something I am proud of because it works end to end on free tiers. Mention the cold start mitigation: Vercel idles functions after five minutes, so I set up a free external cron to ping the health endpoint every two minutes. This keeps the app feeling fast without paying for Vercel Pro. Then walk through the five sprints in one line each.*

Slide 6: Lessons Learned and Thanks

Title: What I Learned and What Comes Next

On the slide (lessons, three or four bullets):

- Deploy on day two, not week five. Cold debugging a stack you have never deployed against is painful.
- Read the free tier fine print before you integrate. Free does not always mean useful.
- For serverless, pick HTTP APIs over SMTP. Outbound TCP is unreliable.
- Strict TypeScript plus shared Zod schemas paid back the setup cost in week one.

On the slide (what is left):

- Write tests (vitest is wired, directories are empty)
- Wire GitHub Actions CI
- Push notifications and scorecard PDF export
- Sentry for error tracking

On the slide (thanks):

- Mohan Ram C for the mentorship and the sector primer idea that shaped the AI advisor design
- IIIT Bangalore for the platform
- Q and A

Speaker note (about 90 seconds):

> *Be honest about what is not done. No tests. No CI. Both go in the roadmap. Then close with a real thank you to your mentor and offer to take questions. If the audience is engaged, this is where the conversation gets interesting.*

Tips for Delivering the Presentation

- Keep slide text short. The audience should listen to you, not read at you.
- Spend most of the time on slide 4 (the demo). The video is the strongest part of the project.
- Practice running the demo video at least twice before the real presentation. Know where to pause and add commentary.
- If the demo video link does not load on the day, have the APK installed on your own phone as a backup and project that screen.
- Be honest about the gaps (no tests, no CI). Supervisors respect honesty more than false polish.
- Have the GitHub link and the handover checklist ready in a browser tab in case anyone asks for code or documentation during questions.

End of document. Saanvi Vishal, 18 May 2026.